Projet E5 Déploiement de trois applications sur Kubernetes

Sahki Enzo Travail individuel 4 novembre 2026 Empreinte des composants : s-04-11-2026 Compte Docker Hub : etest15

Objectif et état du projet

Le but est de déployer trois applications existantes sur Kubernetes, puis de retrouver la même infrastructure dans trois environnements. Une application doit utiliser Stripe. Deux applications doivent être accessibles depuis l'extérieur du cluster et la troisième doit rester interne. Le projet comprend aussi la supervision et l'adaptation automatique du nombre de pods.

Les applications sont déployées dans une VM de travail. Le détail des résultats réellement vérifiés figure dans RESULTATS.md et dans les journaux du dossier logs. La page de l'application Stripe peut être testée sans clés, mais le paiement ne peut être validé qu'après configuration du compte Stripe de test. Aucun paiement réussi n'est affirmé sans preuve.

Organisation du travail

Je suis seul sur le projet. La répartition correspond donc à mes étapes : lecture du sujet, choix des applications, préparation de la VM, création des images, configurations Kubernetes, tests réseau, tests de charge, supervision et rédaction. Les commits Git enregistrent les étapes réellement réalisées. Il n'y a pas de répartition entre plusieurs membres.

Choix des solutions

K3s est une distribution légère de Kubernetes. Elle fournit notamment le moteur de conteneurs containerd, le DNS du cluster, Traefik et metrics-server. Elle permet de travailler avec les objets Kubernetes habituels sans installer chaque composant séparément. En revanche, une seule VM ne permet pas de continuer le service si cette VM s'arrête. Le nom prod représente ici un environnement de TP, pas une plateforme de production complète.

Multipass fournit une VM Ubuntu 24.04 séparée des autres travaux. La VM e5-s-04-11-2026 dispose de 6 processeurs virtuels, 10 Go de RAM et 35 Go de disque. Docker sert à construire et publier les images. Les conteneurs Kubernetes sont ensuite exécutés par containerd, et non par Docker. Importer une image avec k3s ctr permet de tester localement avant sa publication Docker Hub.

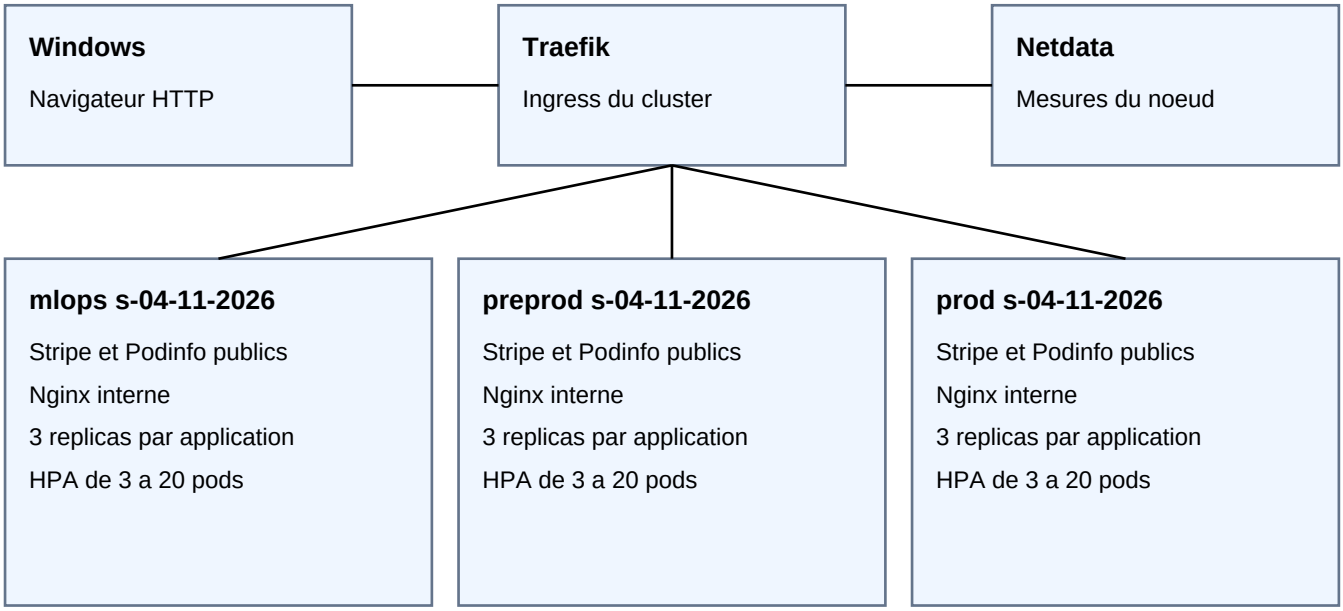
Stripe Checkout provient du dépôt officiel stripe-samples/checkout-one-time-payments. Le serveur Python utilise Flask. Le serveur et le client HTML sont réutilisés sans modification, avec la licence d'origine. Gunicorn démarre le serveur dans le conteneur. Il n'est pas nécessaire d'ajouter Django puisque l'application officielle fournit déjà le paiement demandé. Le choix de Flask vient donc de l'application réutilisée. Cette démonstration n'est pas une boutique complète avec stock, utilisateurs et commandes.

Podinfo est une petite application déjà développée pour les démonstrations Kubernetes. Elle permet de vérifier le routage, les réplicas et la charge. Nginx est un serveur web existant. Sa page par défaut suffit à vérifier un service interne ; aucun site personnalisé n'a été programmé.

Netdata fournit une interface web de supervision. Les montages en lecture seule de /proc et /sys donnent accès à des mesures de l'hôte. Les capacités système permettent des collectes complémentaires. Ce sont des droits importants, limités ici à la VM de TP. Cette installation simple supervise le nœud et les ressources visibles ; elle ne remplace pas une installation complète de supervision Kubernetes multi-nœuds et d'historisation des logs.

Les images de départ sont Python 3.12 slim, Podinfo 6.9.2, Nginx 1.28 alpine et Netdata 2.6.3. Les versions observées et les résultats de construction sont présents dans les journaux. K3s a été installé depuis son canal stable ; sa version effective est consignée dans le journal des nœuds.

Architecture



Le poste Windows accède à l'adresse de la VM en HTTP. Traefik lit le nom demandé dans l'en-tête Host et choisit le service. Dans chaque namespace, les services Stripe et Podinfo sont reliés à un Ingress. Le service Nginx reste en ClusterIP. Chaque application possède un Deployment avec trois réplicas au repos et un HPA limité à vingt réplicas.

Le schéma architecture.svg fourni dans le projet représente ce fonctionnement. Les trois namespaces applicatifs sont mlops-s-04-11-2026, preprod-s-04-11-2026 et prod-s-04-11-2026. Netdata utilise observability-s-04-11-2026. Les composants applicatifs portent aussi le label nom: sahki et l'empreinte s-04-11-2026. Les composants système gérés par K3s gardent les noms prévus par la distribution.

Préparation et construction

Le script 00-vm crée la VM depuis Windows et transfère le projet. Il ne faut pas le relancer si la VM existe déjà. Le script 01-installer installe Docker, curl, Git, ApacheBench et K3s. Une attente vérifie que le nœud est enregistré puis prêt.

Le script 02-images construit les trois images portant l'empreinte. Pour Stripe, il ajoute uniquement le serveur officiel, les fichiers statiques et les dépendances. Les deux autres Dockerfile se basent directement sur les images existantes. Les images sont exportées avec docker save puis importées dans containerd avec k3s ctr images import.

Le script 03-publier utilise docker login puis docker push. Les images finales attendues sont etest15/stripe-s-04-11-2026:1, etest15/podinfo-s-04-11-2026:1 et etest15/nginx-s-04-11-2026:1. Les dépôts Docker Hub doivent être publics pour que le correcteur puisse télécharger les images. Une construction locale ne constitue pas une preuve de publication.

Déploiement et composants Kubernetes

Un Namespace rassemble les objets d'un environnement. Un Deployment demande l'exécution des réplicas et recrée les pods arrêtés. Un Pod contient ici un conteneur applicatif. Le Service fournit une adresse stable devant les pods. L'Ingress donne un point d'entrée HTTP extérieur au cluster. Un Secret fournit les valeurs Stripe sans les écrire dans les manifests versionnés.

La readinessProbe vérifie si un pod peut recevoir du trafic. La livenessProbe vérifie si l'application répond encore. Les requêtes de CPU et mémoire aident Kubernetes à réserver de la capacité ; les limites empêchent une application de consommer sans borne. Un HPA modifie le nombre de réplicas selon les mesures de CPU remontées par metrics-server.

Le script 04-secrets crée les secrets à partir du fichier local .env. Le script 05-deployer applique les mêmes types d'objets dans les trois environnements. Seuls le namespace et le nom d'accès extérieur changent. Le script 06-dockerhub remplace les références locales par les images du compte Docker Hub après leur publication. Après mise à jour d'un Secret, un rollout restart est nécessaire puisque les valeurs sont injectées au démarrage du conteneur.

Les fichiers YAML sont volontairement simples et répétés pour rester faciles à lire. Il n'y a pas de développement d'application ni de modèle Helm personnalisé. Les explications sont dans les documents ; les fichiers de configuration ne contiennent que leur contenu utile.

Isolation des environnements

Un namespace seul ne bloque pas le trafic entre pods. La NetworkPolicy limite donc les connexions entrantes aux pods du même environnement. Une exception autorise les pods Traefik du namespace kube-system vers les ports 4242 et 9898. Nginx écoute sur 80 et ne bénéficie pas de cette exception.

Les sorties réseau restent autorisées, notamment vers DNS et Stripe. Les secrets sont créés séparément dans chaque namespace. ResourceQuota limite le nombre de pods et les réservations de ressources de chaque environnement. LimitRange fournit des valeurs par défaut aux petits pods de vérification et de charge. Ces mesures isolent le trafic applicatif ; les administrateurs du cluster gardent leurs droits. Pour de vraies équipes, il faudrait aussi distribuer des comptes et des rôles RBAC adaptés.

Mesures et autoscaling

Les ressources initiales sont des hypothèses vérifiées ensuite avec kubectl top. Les mesures au repos ont montré environ 50 à 51 Mio pour Stripe, 12 Mio pour Podinfo et 5 Mio pour Nginx. Les réservations mémoire retenues sont respectivement 96 Mio, 32 Mio et 16 Mio. Les limites mémoire sont 256 Mio, 128 Mio et 64 Mio. Les mesures exactes sont conservées dans logs/mesures-repos-s-04-11-2026.log.

Les réservations CPU sont 50 millicœurs pour Stripe, 25 pour Podinfo et 25 pour Nginx. Les limites sont respectivement 500, 300 et 300 millicœurs. Ces valeurs correspondent à une petite démonstration et doivent être revues si la charge réelle change. Le pourcentage du HPA se calcule par rapport à la réservation CPU : 60 % de 50 millicœurs représente 30 millicœurs moyens par pod.

Le script 07-charge utilise ApacheBench depuis un pod du même namespace, ce qui permet aussi de tester Nginx sans le publier. Il envoie des requêtes GET sur la page d'accueil pendant 180 secondes avec une concurrence de 100. Il n'envoie pas de création de paiement en boucle. Les journaux conservent la charge, les mesures CPU et mémoire, le nombre de réplicas pendant six minutes et l'état final. La descente attend une fenêtre de stabilisation de 120 secondes. Cette fenêtre évite de supprimer des pods dès la première baisse de charge.

Trois applications à vingt réplicas dans chacun des trois environnements représenteraient jusqu'à 180 pods applicatifs. Configurer ce plafond ne garantit pas que la VM puisse supporter leur pleine charge simultanée. Les tests du projet portent sur l'environnement mlops, avec les autres environnements au repos. Les trois environnements utilisent les mêmes réglages.

Vérifications réseau et supervision

Le script 09-verifier vérifie un code HTTP 200 pour Stripe et Podinfo dans chaque environnement. Il vérifie aussi un code 404 pour un nom externe Nginx, l'absence de route étant volontaire. Un pod de test vérifie que Nginx répond dans son namespace et que le service du namespace prod n'est pas joignable depuis mlops. Le journal indique INTERNE_OK et ISOLATION_OK lorsque ces contrôles réussissent.

La supervision est accessible par l'Ingress Netdata. L'API /api/v1/info permet de vérifier que l'agent répond ; la page web permet de consulter ses graphiques. Les noms .test doivent pointer vers la VM via le fichier hosts Windows fourni. Il s'agit d'un accès depuis l'extérieur du cluster sur le réseau de la VM, sans exposition Internet ni certificat TLS.

Paiement Stripe de test

Dans Stripe, ouvrir le compte de test ou le sandbox, créer un produit et un tarif, puis relever l'identifiant price_. Renseigner dans .env les clés publique et secrète du même environnement ainsi que PRICE. Appliquer les secrets puis redémarrer Stripe dans les trois namespaces.

Ouvrir la page Stripe mlops, vérifier que le prix s'affiche et cliquer sur le paiement. Sur la page Checkout de test, utiliser 4242 4242 4242 4242, une date future et un CVC à trois chiffres. Après confirmation, vérifier l'état réussi dans l'interface Stripe du même environnement de test. Conserver une capture anonymisée avec la date, le montant et l'identifiant du paiement. Une page de succès seule ne suffit pas à prouver que le tableau Stripe a été vérifié.

Le webhook n'est pas nécessaire pour constater le paiement dans le tableau Stripe. Si son journal est souhaité, utiliser Stripe CLI en mode test avec un transfert vers /webhook, renseigner le secret whsec_, puis redémarrer les pods avant le test. Sans configuration, ne pas présenter le webhook comme validé. Ne pas conserver de clés dans le rendu ou les captures.

Logs et difficultés rencontrées

Le dossier logs regroupe les constructions Docker, le déploiement, les tests réseau, les mesures, les charges, les événements et les descriptions Kubernetes. Le script 08-preuves collecte aussi les journaux applicatifs et le service K3s. L'historique des commandes exécutées par l'agent est fourni comme journal reconstitué clairement identifié ; les nouvelles commandes personnelles peuvent être enregistrées avec script ou history.

Multipass ne répondait pas depuis le bac à sable initial mais fonctionnait avec les droits adaptés. Docker Desktop était arrêté, d'où l'utilisation de Docker dans la VM. Le premier script d'installation interrogeait Kubernetes avant l'enregistrement du nœud ; une attente a été ajoutée. Les erreurs et corrections sont décrites dans logs/erreurs-s-04-11-2026.md. Les preuves réelles sont conservées, y compris les erreurs initiales.

Préparer l'oral et le rendu

Je dois pouvoir expliquer le chemin navigateur, Ingress, Service puis Pod, la différence entre trois réplicas et vingt réplicas, le rôle des requests CPU et la différence entre namespace et isolation réseau. Je dois aussi comprendre pourquoi les clés Stripe restent hors de Git et pourquoi une image locale ne prouve pas sa présence sur Docker Hub.

Avant la remise, consulter RESULTATS.md, terminer les accès personnels manquants, conserver les preuves du paiement et de Docker Hub, puis vérifier que le rapport correspond aux résultats obtenus. Le sujet impose une remise individuelle et une réservation du créneau d'oral. Les documentations officielles et la provenance exacte de l'application réutilisée sont regroupées dans SOURCES.md.

Résultats et vérifications

Résultats vérifiés le 4 novembre 2026

VM Multipass e5-s-04-11-2026 créée : Ubuntu 24.04.5 LTS, 6 vCPU, 10 Go RAM, 35 Go disque. IP observée : 172.26.141.177.

K3s opérationnel : v1.36.5+k3s1, un nœud Ready. Trois images construites, trois environnements déployés, neuf Deployments applicatifs avec trois réplicas au repos, soit 27 pods applicatifs. Netdata fonctionne dans un namespace distinct.

HTTP : les pages Stripe et Podinfo répondent 200 dans mlops, preprod et prod. Le routage Podinfo depuis Windows vers la VM répond également 200. Nginx externe répond 404 ; un pod de mlops peut joindre Nginx en interne et ne peut pas joindre Nginx de prod. Preuve : logs/verification-s-04-11-2026.log. Netdata répond via son API : logs/netdata-info-s-04-11-2026.json.

Autoscaling : les trois applications de mlops sont passées de 3 à 20 réplicas sous charge, puis sont revenues à 3 après l'arrêt de la charge. Les autres environnements sont restés à 3. Preuves : logs/autoscaling-mlops-*.log et logs/retour-mlops-*.log. Le script de test collecte les mesures pendant six minutes pour chaque application.

Charge de 180 secondes, concurrence 100, exécutée simultanément sur les trois applications de mlops : Stripe 942079 requêtes, 5233,60 requêtes/s, 0 erreur ApacheBench ; Nginx 1622264 requêtes, 9012,51 requêtes/s, 0 erreur ApacheBench ; Podinfo 1533802 requêtes, 8521,06 requêtes/s. Le premier test Podinfo compte 1020291 différences de longueur, sans erreur de connexion ou de réception. Les pages Podinfo contiennent des valeurs propres au pod ; la taille peut varier. Le script est corrigé avec ab -l et le résultat du test complémentaire est conservé dans logs/charge-podinfo-correction-s-04-11-2026.log. Ces débits correspondent à ce TP et ne sont pas des garanties de performance.

Stripe : seule la page d'accueil et sa charge HTTP sont vérifiées à ce stade. PRICE=price_non_configure et des clés vides permettent l'aperçu, pas un paiement. Configuration des vraies clés de test et preuve du paiement dans le tableau Stripe à terminer. Aucune réussite de paiement n'est inventée.

Docker Hub : compte prévu etest15. Les images sont construites et importées localement. La connexion et la publication publique doivent être confirmées avant de considérer cette exigence comme terminée.

Git : dépôt local réel avec commits. Le dossier .git est conservé dans le rendu. Aucun historique antérieur fictif ni dépôt distant n'est revendiqué.

Configuration : les YAML, Dockerfile, scripts et exemples de variables sont sans commentaires dièse. Les sources originales de Stripe, dont les commentaires Python et les couleurs CSS, sont préservées avec leur licence. Le code métier n'a pas été développé pour ce projet.

Éléments personnels à terminer : configurer les accès de test, vérifier le paiement et les dépôts Docker Hub, relire le rapport, réserver l'oral et déposer le rendu individuel dans Teams. Les domaines .test nécessitent les lignes du fichier ACCES-WINDOWS.txt dans hosts pour un accès par navigateur depuis Windows.

Sources officielles

Applications réutilisées

Stripe Checkout : https://github.com/stripe-samples/checkout-one-time-payments

Commit téléchargé : c834ba3ae155bd77a6aeb5879b58edd6a944af00

Fichiers Python et client HTML repris sans modification. Licence MIT conservée.

Flask est utilisé par l'exemple officiel. Django n'est donc pas nécessaire.

Podinfo : https://github.com/stefanprodan/podinfo

Nginx : https://hub.docker.com/_/nginx

Netdata : https://learn.netdata.cloud/docs/netdata-agent/installation/docker

K3s : https://docs.k3s.io/quick-start

Réseau K3s : https://docs.k3s.io/networking/networking-services

HPA : https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/

NetworkPolicy : https://kubernetes.io/docs/concepts/services-networking/network-policies/

Multipass : https://canonical.com/multipass/docs/

Stripe test : https://docs.stripe.com/testing

Les commentaires présents dans les sources originales ne sont pas des commentaires de configuration. Les fichiers de configuration et scripts du projet ne contiennent pas de commentaires ajoutés.